Vapor Stone
RAIL SYSTEMS
A Wabtec company

10 655 Henri-Bourassa Blvd., West
Saint-Laurent (Québec) Canada
H4S 1A1

Tel: 514.335.4200
Fax: 514.335.4231

www.wabtec.com

Vapor Stone Rail Systems

Generic Software Documentation Project

Project # 7260

Software Design Description (SDD)
Self-Test

Contract Number		CDRL Number	
		Customer Ref. Number	

NOTICE
Plans, specifications and engineering data submitted herewith or which may be submitted, remain the property of **Vapor Stone Rail Systems** and shall not be reproduced or altered or used in whole or in part except by express consent of **VSRS**.

AVIS
Les plans, spécifications et données techniques soumis ci-joints, ou qui peuvent être soumis, restent la propriété de **Vapor Stone Rail Systems** et ne doivent pas être reproduits, altérés ou utilisés, totalement ou partiellement, sans le consentement express de **VSRS**.

	Name / Title	Signature / Date
Prepared :	Thomas Marmeth Software Designer	2011-10-07
Verified :	Eric Malo, Eng. System Engineer	2011-10-07
Approved :	Martin Ouellet, Eng. Software Department Supervisor	2011-10-11

Document No.	**201111584**	Revision	**1**
		Date (yyyy-mm-dd)	**2011-10-07**

Revision List

Rev.	ECD#	Date	By	Description of changes
1	---	2011-10-07	Thomas Marmeth	Released version.
0.1	---	2011-10-04	Thomas Marmeth	Implemented comments from inspection I-1808
0	---	2011-08-26	Thomas Marmeth	Original issue

Preface

This Software Design Description (SDD) complies with the requirements of the IEEE 1016-1998 and IEEE Std 1558-2005, IEEE Recommended Practice for Software Documentation for Rail Equipment and Systems approved on 2005-05-23.

Table of Contents

Revision List 2
Preface 3
Table of Contents 4
List of Figures 5
List of Tables 5

1 INTRODUCTION 6

1.1.1 Purpose 6
1.1.2 Scope 6
1.1.3 Definitions and Acronyms 6
1.1.4 Acronyms 7
1.1.5 Document Organization 7

2 REFERENCES 9

3 SOFTWARE DESIGN 10

3.1 Decomposition Description 10
3.1.1 Self Test Module Description 10
3.1.2 Concurrent Process Decomposition 10
3.1.3 Data Decomposition 11
3.2 Dependency Description 12
3.2.1 Inter-Module Dependencies 12
3.2.2 Inter-Process Dependencies 12
3.2.3 Data Dependencies 12
3.3 Interface Description 13
3.3.1 Self Test functions 13
3.4 Detailed Design 15
3.4.1 Self-test Module Design 15
3.4.2 Data Entities Detailed Design 25

APPENDIX A 26

List of Figures

Figure 3-1: Inter-Module Dependencies Diagram 10
Figure 3-2: SELFTEST_Execute 15
Figure 3-3: SELFTEST_TaskMemory 17
Figure 3-4: Target Application CRC and Boot CRC tests 20
Figure 3-5: Self-Test RAM Address Test 22
Figure 3-6: Self-Test RAM Pattern Test 24

List of Tables

Table 3-1: Structure of MemBackupZone for pattern and address test verification 11
Table 3-2: Function to Calculate the CRC16 19
Table 3-3: CRC Look-Up Table for CRC16 Calculation 19
Table 3-4: Memory Test Structure of SRAM and NVRAM 21
Table A-1: Memory Map of CRC information of Boot Loader 26
Table A-2: Memory Map of CRC information of Target Application 26

1 INTRODUCTION

This document is the Software Design Description (SDD) of the Self Test Module developed by Vapor Stone Rail Systems.

The Self Test Module can be part of any of the Door Module System (DMS) Applications.

1.1.1 Purpose

The purpose of this SDD is to describe the software components and implementation details of the Self Test Module.

The intended audience is the Vapor Stone Rail Systems software development team and maintenance team members of the transit authority and the car builder.

1.1.2 Scope

The Self Test Module is a software module of the Door Module System (DMS) Application.

1.1.3 Definitions and Acronyms

1.1.3.1 Definitions

Boot Loader	Software starting the Door Control Module application.
Car Network	Data bus and optionally trainlines connecting the Door Control Module with the rest of the train (others Door Control Modules in the car and others cars).
Communication Gateway Module	A specific processor based electronic module running a Communication Gateway Module Application. It acts as gateway between the Door Control Module Application and the Car Network/Portable Test Application.
Door Module System	A term that is used for generically identifying a processor based electronic module running any software within any Door Control Unit.
Door Control Unit	The final unit that is used to control one or more door panels. This unit is normally made of one or two door control modules running the Door Control Module Application and possibly one Communication Gateway Module Application.
Door Control Module	A specific processor based electronic module running a Door Control Module Application. It is used for controlling the door motor.
HW_COM	Communication interface used for communication between Portable Test Application and Door Control Module application, that can be USB, serial or Ethernet.

1.1.3.2 Event Level Definitions

Event:	Term used to globally identify an Info, an Exception or a Failure.
Info:	Info is an occurrence of an Informative condition monitored by the system, which is neither an exception nor a failure.
Exception:	An exception is an occurrence of a condition that is outside of the specified parameters and affecting an outside Line Replaceable Unit (LRU).

Failure: A failure is an occurrence of a condition that is outside of the specified parameters and affecting this Line Replaceable Unit. A failure normally requires maintenance or replacement to restore affected unit to its normal operation conditions.

1.1.4 Acronyms

ASCII	American Standard Code for Information Interchange
BL	Boot Loader
CCITT	Comité Consultatif International Téléphonique et Télégraphique
CDRL	Contract Data Requirements List
CDS	Car Diagnostic System
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
DCM	Door Control Module
DMS	Door Module System
ECD	Engineering Change Document
HW	HardWare
IEEE	Institute of Electrical and Electronics Engineers
IO	Input Output
IRAM	Internal (static) Random Access Memory
KB	Kilobytes
LED	Light Emitting Diode
LH	Left Hand
LRU	Line Replaceable Unit
MHz	MegaHertz
ms	millisecond
NVRAM	Non-Volatile Random Access Memory
OS	Operating System
PTA	Portable Test Application
RAM	Random Access Memory
RH	Right Hand
RTC	Real-Time Clock
SCI	Software Configuration Items
SDD	Software Design Description
SRAM	Static Random Access Memory
SRS	Software Requirements Specifications
Std	Standard
TS	Technical Specification
VSRS	Vapor Stone Rail Systems
XRAM	eXtension Random Access Memory

1.1.5 Document Organization

The Software Design Description (SDD) document describes the architectural software design of the DMS Application Self Test Module. This SDD is divided into 3 sections and the following specific information is provided within each one:

- Section 1, Introduction, states the purpose, identifies the scope, gives definitions, abbreviations and an overview of the SDD.
- Section 2, References, lists the factors that affect the product and its requirements.
- Section 3, Software design summarizes followings:
    - Decomposition Description: describes software entities.
    - Dependency Description: gives a brief description of the dependency between each entity.
    - Interface Description: describes the function interface for each entity.

- Detailed Design: details the algorithm of functions implemented by each software entity, presents and explains tasks performed by each software entity in functional conditions.

Within the document, diagrams, flowcharts and tables are used to present the Self Test Module software design:

- Decision flowcharts give a graphical view of how entities and functions process work.
- Figure 3-1 describes the dependencies between different modules.
- Tables contain data structures used by the Self Test Module. They also present C code.

2 REFERENCES

This section lists the number, title, revision, date, and source of all documents referenced in this SDD.

Standards

IEEE Std 1016-1998	IEEE Recommended Practice for Software Design Descriptions, SePTAmber 23rd 1998, IEEE, www.ieee.org
IEEE Std 1558-2005	IEEE Recommended Practice for Software Documentation for Rail Equipment and Systems, March 23rd 2005, IEEE, www.ieee.org
ITU-T X.25	International Telecommunication Union Telecommunication Standardization Sector Recommendation X.25

Vapor Stone Rail Systems Documents

S17-07	Software Design Description Template, Rev. 0, 2002-01-15 complemented with requirements from IEEE Std 1558.
CMM-01	Software Engineering Process, Rev. 3, 2001-03-31
CMMI-02	Software Engineering Policy Manual, Rev. 1, 2008-07-18
SACS	Software Applications Configuration Specifications (SACS), Project specific
FMP	Fault Management Plan (FMP), Project specific

3 SOFTWARE DESIGN

3.1 Decomposition Description

The Self-Test module is a single entity which provides two functionalities:

- A self-test status request.
- A continuous self-test.

Modules interaction is described in Figure 3-1.

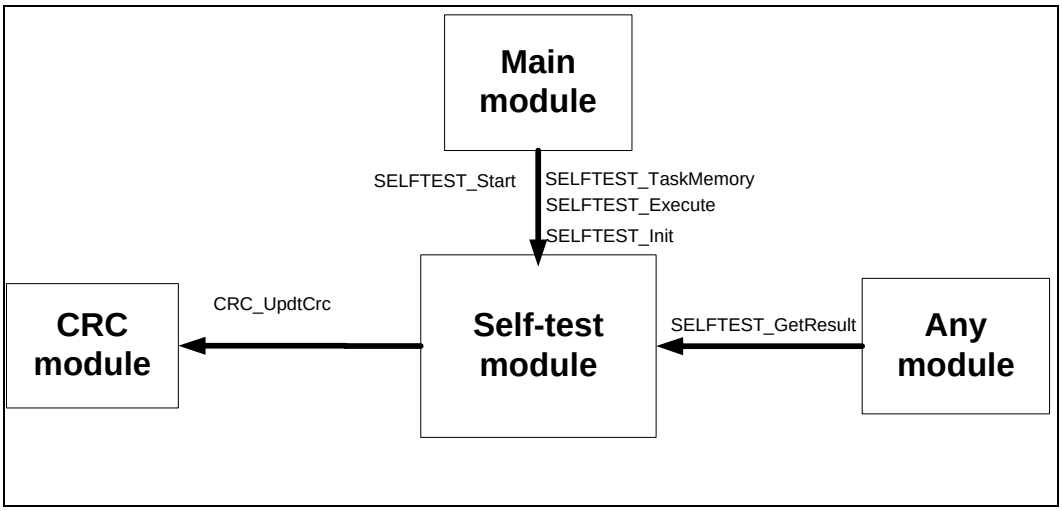

Figure 3-1: Inter-Module Dependencies Diagram

3.1.1 Self Test Module Description

The Self-Test status results can be retrieved by any module using function SELFTEST_GetResult.

3.1.2 Concurrent Process Decomposition

3.1.2.1 Self Test Tasks Description

Continuous Self-Test is a low priority task scheduled at a low periodicity. The following describes the steps executed by a continuous Self-Test :

- Verifies the integrity of the memory
- Verifies the integrity of the program

The module functions shall be designed to support multi-threading.

3.1.3 Data Decomposition

3.1.3.1 Self Test

This section shows the structure used by selftest module to backup the memory location when performing test in SRAM and NVRAM. An union structure is used and divided in 2 parts. First part of the union is used whenever the address lines test is in progress, to backup data on memory cells to be tested. The second part of the union is used when the pattern test is in progress. The structure used in detailled in Table 3-1.

Table 3-1: Structure of MemBackupZone for pattern and address test verification

Identification	MemBackupZone
Entity Type	struct RamTest
Prototype	

```
typedef struct _RamTest
{
typedef union
{
    struct
    {
        unsigned char ucNotUsed1;     // Located @ 0xA7F00
        unsigned char ucCommand;      // Located @ 0xA7F01
                                      // Contain 0x01 for Address lines
                                      // Contain 0x02 for Pattern test
        unsigned char ucComplCommand; // Located @ 0xA7F02
                                      // Contain 0xFE for Address lines
                                      // Contain 0xFD for Pattern test
        unsigned char ucNotUsed2;     // Located @ 0xA7F03
        unsigned int  uiCrc;          // Starts @ 0xA7F04 (2 bytes)
        unsigned char ucNbData;       // Located @ 0xA7F06
        unsigned char ucData[22];     // Starts @ 0xA7F07 (2 bytes)

    }AddressLine;

    struct
    {
        unsigned char ucNotUsed1;     // Located @ 0xA7F00
        unsigned char ucCommand;      // Located @ 0xA7F01
                                      // Contain 0x01 for Address lines
                                      // Contain 0x02 for Pattern test
        unsigned char ucComplCommand; // Located @ 0xA7F02
                                      // Contain 0xFE for Address lines
                                      // Contain 0xFD for Pattern test
        unsigned char ucNotUsed2;     // Located @ 0xA7F03
        unsigned int  uiCrc;          // Starts @ 0xA7F04 (2 bytes)
        unsigned long ulAddress;      // Starts @ 0xA7F06 (4 bytes)
        unsigned int  uiData;         // Starts @ 0xA7F0A (2 bytes)

    }Pattern;

}RamTest;

RamTest far MemBackupZone;
```

3.2 Dependency Description

3.2.1 Inter-Module Dependencies

3.2.1.1 Self Test Module

Figure 3-1 shows the Inter-Module Dependencies Diagram of the Self Test Module.

3.2.2 Inter-Process Dependencies

Not applicable.

3.2.3 Data Dependencies

Not applicable.

3.3 Interface Description

This section describes the functions of the Self Test Module. The function name, a description of the function, input and output parameters required by the function are provided for each function of each module.

3.3.1 Self Test functions

The Self-Test module implements the following functions (sorted alphabetically):

3.3.1.1 Self Test Module External Interfaces

Function Name	unsigned char SELFTEST_Execute (unsigned char ucTestNumber)
Description	Executes one self-test.
Input	unsigned char ucTestNumber: One of the following to return the corresponding test result. DCM_CRC_SELFTEST BOOT_CRC_SELFTEST SRAM_PATTERN_SELFTEST SRAM_ADDR_SELFTEST NVRAM_PATTERN_SELFTEST NVRAM_ADDR_SELFTEST
Return	SELFTEST_SUCCESS SELFTEST_FAIL SELFTEST_IN_PROGRESS

Function Name	unsigned int SELFTEST_GetResult (void)
Description	Sends the last stored self test result.
Input	None
Return	uiStatus, containing the result of the last self-test : SELFTEST_FAIL if the test failed SELFTEST_SUCCESS if the test is success

Function Name	void SELFTEST_Init (void)
Description	Initializes the Self-test driver.
Input	None
Return	None

Function Name	void SELFTEST_Start (void)
Description	Start the self-test task
Input	None
Return	None

Function Name	void SELFTEST_TaskMemory (void)
Description	Task that calls each self-test. One test or a part of it (for RAM pattern test) is executed every minute.
Input	None
Return	None

3.3.1.2 Self Test Module Internal Interfaces

Function Name	unsigned char SELFTEST_FlashAppCrc (void)
Description	Executes a self-test for the application portion of Flash Memory
Input	None
Return	Result of the test : SELFTEST_FAIL if the test failed SELFTEST_IN_PROGRESS if the test is in progress SELFTEST_SUCCESS if the test is success

Function Name	unsigned char SELFTEST_FlashBootCrc (void)
Description	Executes a self-test for the BL portion of the Flash memory
Input	None
Return	Result of the test : SELFTEST_FAIL if the test failed SELFTEST_IN_PROGRESS if the test is in progress SELFTEST_SUCCESS if the test is success

Function Name	unsigned char SELFTEST_RamAddressLine (unsigned long ulBase, unsigned long ulSize)
Description	Executes a self-test on Address/Data line for the SRAM
Input	unsigned long ulBase: Base address unsigned long ulSize: Size of the memory
Return	Result of the test : SELFTEST_SUCCESS SELFTEST_FAIL

Function Name	unsigned char SELFTEST_RamPattern (void)
Description	Executes a pattern test on the RAM
Input	None
Return	SELFTEST_FAIL if the test failed SELFTEST_SUCCESS if the test is success SELFTEST_IN_PROGRESS if the test is in progress

Function Name	bit SELFTEST_TestInProgressCheck (void)
Description	Verify if previous test was in progress.
Input	None
Return	bit bStatus: TRUE if test backup area works, FALSE otherwise.

3.4 Detailed Design

3.4.1 Self-test Module Design

The Self-Test module provides two functionalities:

- a Self-Test execution at startup (SELFTEST_Execute, punctual self-test).
- a Continuous Self-Test (SELFTEST_TaskMemory).

The startup Self-Test tests the adresses of the SRAM and NVRAM before allowing the software to run. Continuous Self-Test is a task with a low priority and scheduled at a low periodicity (each second) that verifies the integrity of the memory and the program.
The startup self-tests use the SELFTEST_Execute fonction described by Figure 3-2 while Figure 3-3 shows the sequence to process a Continuous Self-Test.

3.4.1.1 Ponctual Self-Test (SELFTEST_Execute)

The module Self-Test executes the requested self-test. The Figure 3-2 shows the processing of a request. The module executes the requested test and analyzes the result. An error is logged when a test fail. The corresponding error is reset when the test passes. The following faults are logged by this module:

- Target Application CRC Discrepancy Detected
- Boot Loader CRC Discrepancy Detected
- SRAM Failure Detected
- NVRAM Failure Detected

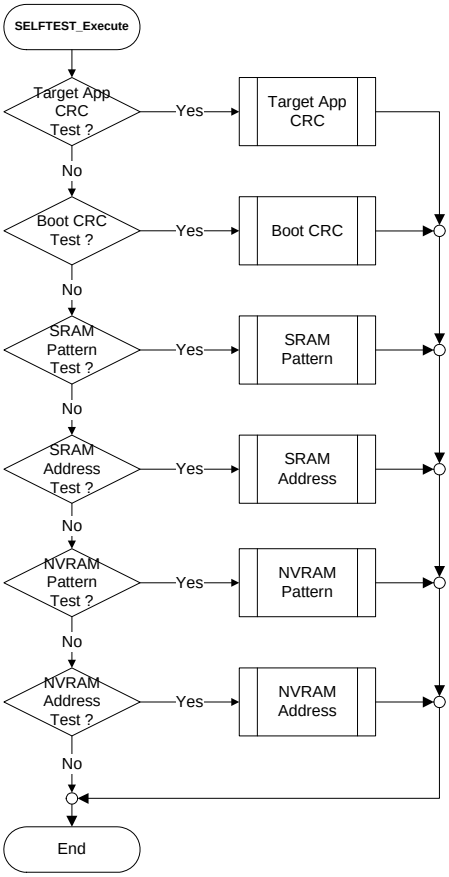

Figure 3-2: SELFTEST_Execute

3.4.1.2 SELFTEST_GetResult

This function returns the result of the last executed self-test.
This function is too simple to provide an algorithm/flowchart.

3.4.1.3 SELFTEST_Init

Initialize all variables used for the 6 self-tests.
This function is too simple to provide an algorithm/flowchart.

3.4.1.4 SELFTEST_Start

Start Self-Test Task.
This function is too simple to provide an algorithm/flowchart.

3.4.1.5 Continuous Self-Test task (SELFTEST_TaskMemory)

This test is partially executed once every second. The verification of the integrity of the memory is synchronized with the verification of the integrity of the program application so that they end at the same time.
The task Continuous Self-Test executes all self-tests. In case of fatal error as CRC error on target application Boot loader or Target Application, SRAM or NVRAM memory error (pattern or address lines) the task logs the error and then resets the CPU to restart in the Boot Loader application. See Figure 3-3 for complete description of the task.
The following faults are logged by this module:

- Target Application CRC Discrepancy Detected
- Boot Loader CRC Discrepancy Detected
- SRAM Failure Detected
- NVRAM Failure Detected

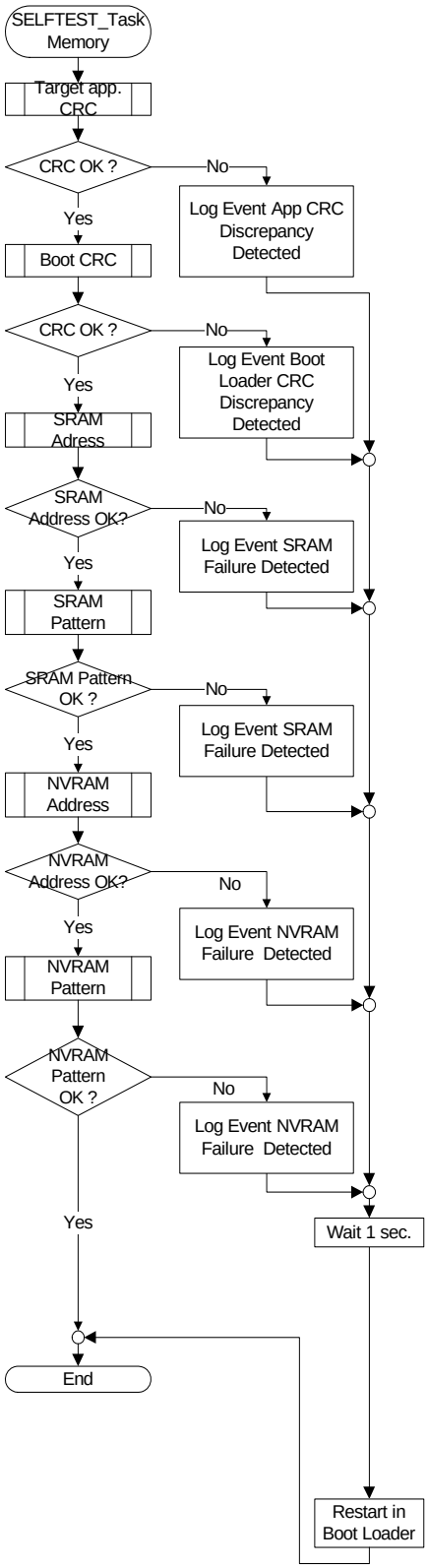

Figure 3-3: SELFTEST_TaskMemory

The following sections describe how each test is performed.

3.4.1.6 Integrity of the Program (SELFTEST_FlashAppCrc, SELFTEST_FlashBootCrc)

The Self-Test for the verification of the integrity of the program is performed by calculating the CRC of the Target Application. To verify the Flash Memory where the program application resides, 16 bytes of the program application is verified by calculating the CRC. After the whole program application is done, the CRC value is verified. If there is a discrepancy, a fault is generated and the program resets by executing Boot Loader Application.

The integrity of the Target Application code is verified by calculating the CRC on the code stored in flash memory. The CRC, the code length, and the revision number are included in the binary file. Table A-1 and Table A-2 show the location of data stored in the flash memory. The CRC is calculated according to the 16-bits CRC-CCITT Standard of document ITU-T X.25 (see Table 3-2 and Table 3-3 for details how to calculate the CRC). Figure 3-4 shows the flow chart of the module that verifies the CRC of Target Application and the Boot Loader Application.

A pre-computed CRC table is used to accelerate CRC calculation and only calculates index of value in table. The initial “uiOldCrc” value is 0 in the algorithm of Figure 3-4:

Table 3-2: Function to Calculate the CRC16

```
/*****************************************************************************/
//  PURPOSE         : This function calculates the CRC16 of the DATA received.
//  INPUT(S)        : ucNewData = Is the new data to add on the old crc. The //
                  initial value to calculate crc is 0.
//                      uiOldCrc  = Is the old crc value to add.
//  RETURN          : The new crc calculated with the new data received.
/*****************************************************************************/
unsigned int CRC_UpdtCrc (unsigned char ucNewData, unsigned int uiOldCrc)
{
    return (crctab[(uiOldCrc >> 8)^ucNewData] ^(uiOldCrc << 8));
}
```

Table 3-3: CRC Look-Up Table for CRC16 Calculation

```
const unsigned int near crctab[] = {
     0x0000,  0x1021,  0x2042,  0x3063,  0x4084,  0x50A5,  0x60C6,  0x70E7,
     0x8108,  0x9129,  0xA14A,  0xB16B,  0xC18C,  0xD1AD,  0xE1CE,  0xF1EF,
     0x1231,  0x0210,  0x3273,  0x2252,  0x52B5,  0x4294,  0x72F7,  0x62D6,
     0x9339,  0x8318,  0xB37B,  0xA35A,  0xD3BD,  0xC39C,  0xF3FF,  0xE3DE,
     0x2462,  0x3443,  0x0420,  0x1401,  0x64E6,  0x74C7,  0x44A4,  0x5485,
     0xA56A,  0xB54B,  0x8528,  0x9509,  0xE5EE,  0xF5CF,  0xC5AC,  0xD58D,
     0x3653,  0x2672,  0x1611,  0x0630,  0x76D7,  0x66F6,  0x5695,  0x46B4,
     0xB75B,  0xA77A,  0x9719,  0x8738,  0xF7DF,  0xE7FE,  0xD79D,  0xC7BC,
     0x48C4,  0x58E5,  0x6886,  0x78A7,  0x0840,  0x1861,  0x2802,  0x3823,
     0xC9CC,  0xD9ED,  0xE98E,  0xF9AF,  0x8948,  0x9969,  0xA90A,  0xB92B,
     0x5AF5,  0x4AD4,  0x7AB7,  0x6A96,  0x1A71,  0x0A50,  0x3A33,  0x2A12,
     0xDBFD,  0xCBDC,  0xFBBF,  0xEB9E,  0x9B79,  0x8B58,  0xBB3B,  0xAB1A,
     0x6CA6,  0x7C87,  0x4CE4,  0x5CC5,  0x2C22,  0x3C03,  0x0C60,  0x1C41,
     0xEDAE,  0xFD8F,  0xCDEC,  0xDDCD,  0xAD2A,  0xBD0B,  0x8D68,  0x9D49,
     0x7E97,  0x6EB6,  0x5ED5,  0x4EF4,  0x3E13,  0x2E32,  0x1E51,  0x0E70,
     0xFF9F,  0xEFBE,  0xDFDD,  0xCFFC,  0xBF1B,  0xAF3A,  0x9F59,  0x8F78,
     0x9188,  0x81A9,  0xB1CA,  0xA1EB,  0xD10C,  0xC12D,  0xF14E,  0xE16F,
     0x1080,  0x00A1,  0x30C2,  0x20E3,  0x5004,  0x4025,  0x7046,  0x6067,
     0x83B9,  0x9398,  0xA3FB,  0xB3DA,  0xC33D,  0xD31C,  0xE37F,  0xF35E,
     0x02B1,  0x1290,  0x22F3,  0x32D2,  0x4235,  0x5214,  0x6277,  0x7256,
     0xB5EA,  0xA5CB,  0x95A8,  0x8589,  0xF56E,  0xE54F,  0xD52C,  0xC50D,
     0x34E2,  0x24C3,  0x14A0,  0x0481,  0x7466,  0x6447,  0x5424,  0x4405,
     0xA7DB,  0xB7FA,  0x8799,  0x97B8,  0xE75F,  0xF77E,  0xC71D,  0xD73C,
     0x26D3,  0x36F2,  0x0691,  0x16B0,  0x6657,  0x7676,  0x4615,  0x5634,
     0xD94C,  0xC96D,  0xF90E,  0xE92F,  0x99C8,  0x89E9,  0xB98A,  0xA9AB,
     0x5844,  0x4865,  0x7806,  0x6827,  0x18C0,  0x08E1,  0x3882,  0x28A3,
     0xCB7D,  0xDB5C,  0xEB3F,  0xFB1E,  0x8BF9,  0x9BD8,  0xABBB,  0xBB9A,
     0x4A75,  0x5A54,  0x6A37,  0x7A16,  0x0AF1,  0x1AD0,  0x2AB3,  0x3A92,
     0xFD2E,  0xED0F,  0xDD6C,  0xCD4D,  0xBDAA,  0xAD8B,  0x9DE8,  0x8DC9,
     0x7C26,  0x6C07,  0x5C64,  0x4C45,  0x3CA2,  0x2C83,  0x1CE0,  0x0CC1,
     0xEF1F,  0xFF3E,  0xCF5D,  0xDF7C,  0xAF9B,  0xBFBA,  0x8FD9,  0x9FF8,
     0x6E17,  0x7E36,  0x4E55,  0x5E74,  0x2E93,  0x3EB2,  0x0ED1,  0x1EF0
};
```

The modules verifies at first if the code is present by reading the length at address shown in Table A-1 and Table A-2, and then verifies if the value is greater than zero. In case the value read is zero, the execution of the module is aborted and an error code is return. When the length value is not 0, the CRC is calculated for the length of the code except for the location where the CRC itself is stored. When the code is finished to be processed, the calculated CRC is then compared to the one stored. If any discrepancy is found between the CRC calculated and the CRC read, the application logs Event: target Application CRC Discrepancy Detected or Event:Boot Loader CRC Discrepancy Detected, reset itself for the Boot Loader Application to start.

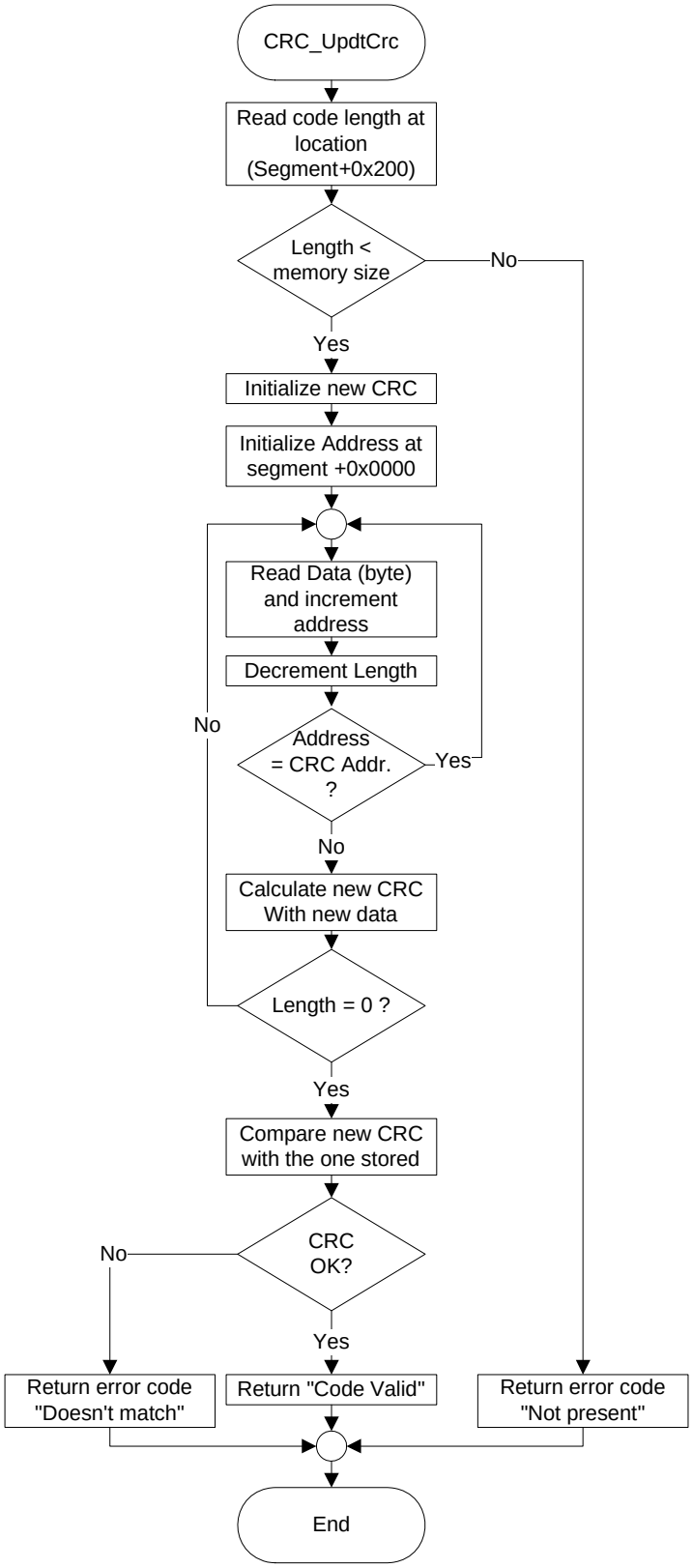

Figure 3-4: Target Application CRC and Boot CRC tests CRC_UpdtCrc

3.4.1.7 SRAM/NVRAM Address Test (SELFTEST_RamAddressLine)

The first step is to verify that each address/data lines are working properly. Writing at each address line a sequential number does this task. The sequential number is then read back and compared to sequential values. Figure 3-5 shows the flowchart used to perform the Self-Test SRAM Address.This test ensures that every address/data lines are responding.

If the test is successful, the pattern test is started. An example of data written in memory is shown in Table 3-4. Before the module starts to write sequential number, a backup of written address data is performed to ensure there is no loss of data in case of a power failure while the test is executed. At the end of the test, the data backed up is restored. If a power failure occurs before the end of the test, during the next start-up, before the test is executed, the module restores the data to preserve the integrity of the RAM.

Table 3-4: Memory Test Structure of SRAM and NVRAM

SRAM Address	Data	NVRAM Address	Data
0x100000	0x01	0x120000	0x01
0x100001	0x02	0x120001	0x02
0x100002	0x03	0x120002	0x03
0x100004	0x04	0x120004	0x04
0x100008	0x05	0x120008	0x05
…….	…….	…….	…….
0x108000	0x11	0x122000	0x0E
0x110000	0x12	0x124000	0x0F

Table 3-1 shows the structure recorded in memory when address lines verification test is started. The first step is to read the data in memory and create a copy of the data starting at address 0x127F05. When the data have been copied, the command 0x01 is written at address 0x127F01 and the complement of the command is written at address 0x127F02. A CRC is calculated on the first two bytes (command and complement) and written into address 0x127F03 and 0x127F04. The memory test is considered as “in progress” after the command, the complement of the command and the CRC have been written. This security is used in case of a power failure to be able to reconstruct the integrity of the memory.
The sequential numbers are written in memory as per data shown in Table 3-1. The sequential numbers are then read back and compared to the sequence. In case of error during the test, the function ends. After the test is completed and successful, the data is then restored to all address location and then the command is canceled by returning the address 0x127F01 to value 0x00.

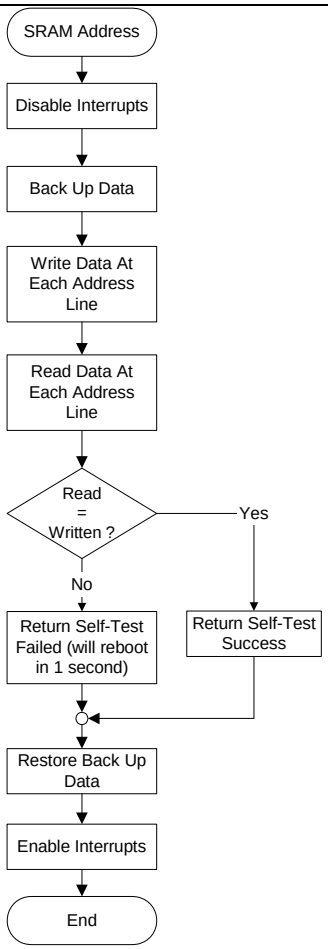

Figure 3-5: SELFTEST_RamAddressLine

3.4.1.8 SRAM/NVRAM Pattern Test (SELFTEST_RamPattern)

The purpose of this test is to verify sequentially every cell of the SRAM or the NVRAM and to verify that each address/data lines works properly by writing and reading values at specific memory location. Figure 3-6 shows the sequence to perform the RAM Pattern Self-Test. If any discrepancy is found between the values written and the values read, the function returns an error code.

To verify each address line, the module is writing at each address a sequential number and then reads it back.
The RAM Pattern test keeps a backup of cells that has to be tested to preserve the integrity of the memory under test and the capability of data recovery. Table 3-1 shows the structure recorded in memory to preserve the integrity of the RAM when pattern test is running.

Upon start of the function, a check is made to see if a previous session was in progress and the data are returned to their respective addresses if this is the case. The integrity of the memory used for the back-up is tested prior to start the test on the SRAM. A quick pattern test is written from address 0x127F00 to 0x127F19. Pattern data 0xAA55 is written and then compared and pattern data 0x55AA is written and compared. At this point if an error occurs, the function ends.

The purpose of the pattern test is to test every bit in the memory. The test consists of two different patterns written in every cell. The first step is to read the data (16 bits) from the memory cell under test and copy it at address 0x127F0A. The address of the cell under test is saved as a long type and written into address 0x127F06 to 0x127F09. When the data has been copied, the command 0x02 is written at address 0x127F01 and the command complement is written at address 0x127F02. A CRC is calculated on the first two bytes (command and complement) and written into address 0x127F03 and 0x127F04. The memory test is considered “in progress” after the command, its complement and the CRC have been written. This security is used in case of power failure to be able to restore memory integrity.

The pattern test writes data 0xAA55 in the memory cell under test and then reads it back to compare the results. In the same memory cell, data 0x55AA is written and then read back to compare. In case of error during the pattern test, the function ends. After the memory cell test is completed and successful, the data is then restored to the address location and the command is canceled by returning the address 0x127F01 to value 0x00.

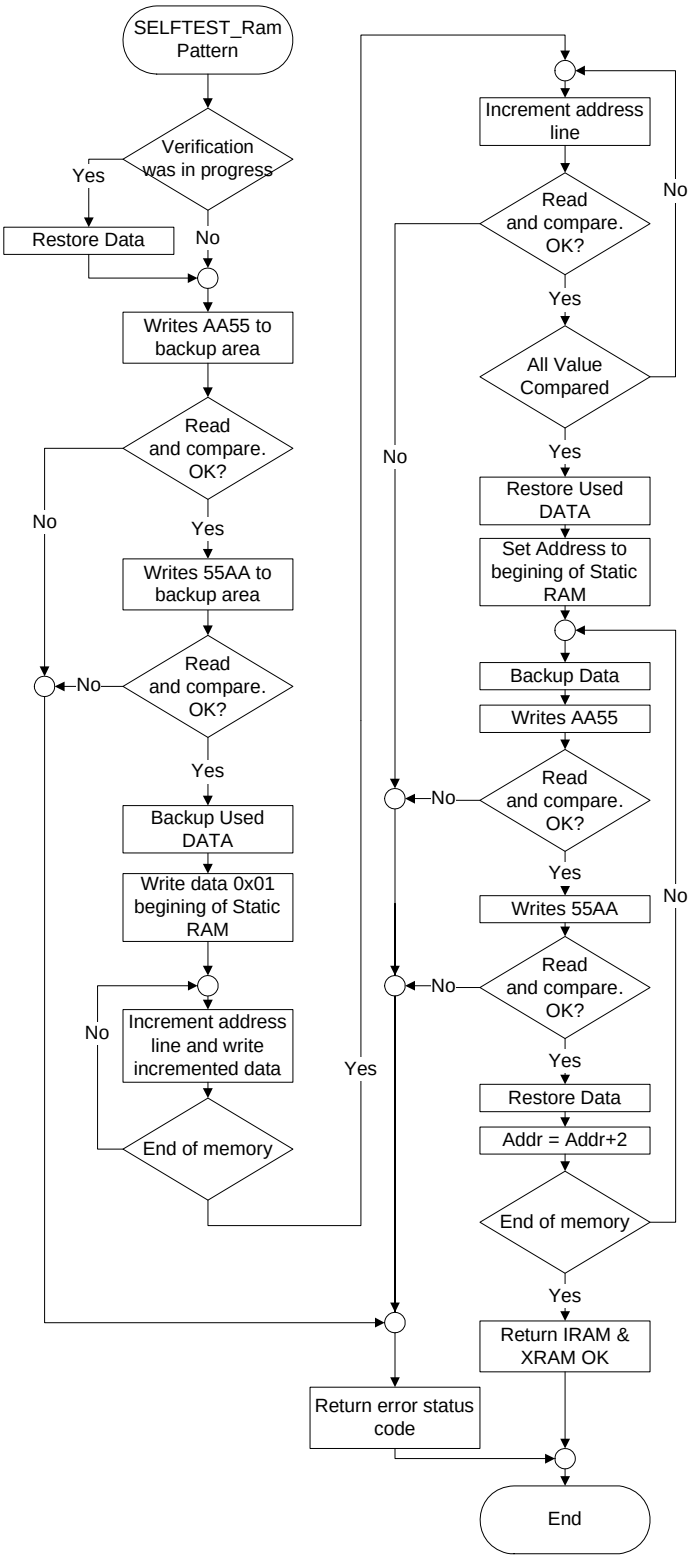

Figure 3-6: SELFTEST_RamPattern

3.4.1.9 SELFTEST_TestInProgressCheck

This function verifies if an address-data line test or a pattern test was in progress in NVRAM. If this is the case, the function will calculate the CRC on all the data, compare it with CRC written in backup zone. If they are found to be equals, the function will restore the data.
This function is too simple to provide an algorithm/flowchart.

3.4.2 Data Entities Detailed Design

Not applicable.

APPENDIX A

The data contained in this appendix is project's specific, the values given hereafter are the default values.

Table A-1 and **Table** A-2 are presenting the memory map of the CRC information for the boot loader and the target application

Table A-1: Memory Map of CRC information of Boot Loader

Address	Length	Description
0x200	4 bytes	Code Size
0x204	2 bytes	CRC
0x206	1 byte	SCI Index: Boot Loader = 4
0x207	1 byte	SCI Type: Software = 1
0x208	8 bytes	Acronym of SCI in ASCII (padded with Null characters if less then 8 characters)
0x210	6 byte	Revision String (padded with Null characters if less then 6 characters)
0x216	11 bytes	Revision Date (Mmm DD YYYY) in ASCII
0x221	15 Bytes	Reserved

Table A-2: Memory Map of CRC information of Target Application

Address	Length	Description
0x20200	4 bytes	Code Size
0x20204	2 bytes	CRC
0x20206	1 byte	SCI Index: DCM Application = 1
0x20207	1 byte	SCI Type: Software = 1
0x20208	8 bytes	Acronym of SCI in ASCII (padded with Null characters if less then 8 characters)
0x20210	6 byte	Revision String (padded with Null characters if less then 6 characters)
0x20216	11 bytes	Revision Date (Mmm DD YYYY) in ASCII
0x20221	15 Bytes	Reserved

Size of the SRAM and NVRAM:
The SRAM is 128KB wide and 17 bits address lines are needed to cover the entire SRAM. The SRAM starts at address 0x100000. The NVRAM is 32KB wide and 15 bits address lines are needed to cover the entire NVRAM. The NVRAM starts at address 0x120000.